CPU (Serial/OpenMP) vs GPU (CUDA) Sorting: Design and Performance Report

March 26, 2026

Abstract

This report presents a performance and engineering comparison of three sorting approaches implemented within one benchmarked application: (1) a serial CPU baseline using a highly optimized standard-library sort, (2) a multi-core CPU parallel merge sort using OpenMP task parallelism with a tunable cutoff for task granularity, and (3) a CUDA GPU bitonic sorting network with safe power-of-two padding for arbitrary input lengths. The goal is to quantify scaling with input size, quantify multi-thread CPU speedup, evaluate GPU tuning sensitivity (block size), and interpret the role of overheads such as synchronization and data movement. Results are based on the measured dataset, and the four plots present in the `results/` directory.

1. Introduction

Sorting is a foundational primitive in systems and performance-critical pipelines: it appears in database query execution, data analytics, scientific computing, and general-purpose applications. Because it is both ubiquitous and sensitive to compute and memory behavior, sorting provides an excellent case study for understanding parallel performance on modern heterogeneous hardware.

Modern platforms typically offer two complementary forms of parallelism:

- **Shared memory CPU parallelism** (multi-core threads accessing the same memory), where task management overhead and memory bandwidth often govern scaling.
- **GPU data parallelism** (many lightweight threads with high throughput), where kernel launch structure and host–device data movement can dominate end-to-end runtime.

This project is designed to make these tradeoffs measurable and explainable. It integrates multiple implementations behind one experimental driver, enforces correctness checking, logs results in a consistent schema, and generates standardized plots that highlight scaling, speedup, and tuning effects.

2. Objectives and Deliverables

The project targets four concrete objectives:

1. **Provide comparable implementations** spanning serial CPU, parallel CPU, and GPU execution models.
2. **Measure performance under controlled variability** by using repeat trials and multiple input distributions.
3. **Separate tuning knobs** relevant to each model (thread count and task cutoff on CPU; block size on GPU).
4. **Produce publication-style figures** that summarize scaling and tuning behavior.

The primary deliverables are:

- A unified benchmarking application that generates inputs, runs each implementation, verifies correctness, and records averaged timings.
- A results dataset (CSV) with consistent columns for analysis.
- A figure set (four plots) summarizes the most important comparisons.

3. Platform Summary

The recorded platform information indicates the following benchmark environment:

- **CPU:** AMD Ryzen 7 5800H (16 logical CPUs)
- **GPU:** NVIDIA GeForce RTX 3050 Laptop GPU (4 GB)
- **Toolchain:** modern GCC toolchain and CUDA 12.x

All numerical results referenced in this report are taken directly from the recorded results dataset and its corresponding plot images.

4. Experimental Methodology

4.1 Input generation and distributions

The benchmark generates integer arrays using a seeded pseudo-random generator and supports four distributions that emphasize different algorithm behaviors:

- **Uniform:** values drawn from a fixed integer range.
- **Gaussian:** values drawn from a normal distribution (then cast to integers).
- **Nearly sorted:** a uniform array is first sorted ascending, then lightly perturbed by a small percentage of random swaps.
- **Reversed:** a uniform array is sorted descending.

4.2 Repeats, averaging, and correctness

Each configuration is executed multiple times (five repeats in the recorded dataset). A new input is generated per repeat by incrementing the base seed, which prevents measurements from being dominated by a single lucky or unlucky array.

Correctness is enforced by validating that the final output is non-decreasing after each run. This is a crucial requirement for parallel algorithms, where races and boundary mistakes can silently corrupt results if not checked.

4.3 What the reported time represents

The dataset's primary metric is **end-to-end average runtime in milliseconds**. End-to-end timing is the right metric for most engineering decisions because it captures all overheads that a real application would pay.

For the GPU implementation, the internal design also records transfer and compute phases; however, the plots and CSV values summarize the overall end-to-end time.

5. Implementation Overview

5.1 Serial CPU baseline (standard-library sort)

The serial baseline uses the C++ standard library to sort on an in-memory integer array. This baseline is intentionally strong: it represents a highly optimized implementation that real applications would use. A credible parallel result should beat this baseline at scale, not merely beat a naïve serial algorithm.

5.2 OpenMP task-parallel merge sort

The CPU-parallel implementation uses a divide-and-conquer merge sort with OpenMP tasks. Conceptually:

- Split the array into two halves.
- Sort each half in parallel as separate tasks.
- Wait for both tasks.
- Merge the two sorted halves using a temporary buffer.

A key tuning parameter is a **cutoff threshold** controlling task granularity. When a subproblem's size falls below the cutoff, the algorithm stops spawning tasks and sorts that segment using a serial sort. This hybrid approach is standard practice because task creation and scheduling overhead can overwhelm the work performed by tiny tasks.

A simplified pseudocode sketch illustrates the structure:

```
parallel region:
  single thread:
    mergesort(left, right)

mergesort(left, right):
  if size <= cutoff:
    serial_sort(segment)
    return
  mid = (left + right)/2
  spawn task mergesort(left, mid)
  spawn task mergesort(mid+1, right)
  wait for tasks
  merge(left, mid, right)
```

5.3 CUDA bitonic sorting network

The GPU implementation uses a bitonic sorting network. Sorting networks are attractive on GPUs because they apply a structured sequence of compare–swap operations that can be mapped to data-parallel kernels.

Because bitonic networks are naturally defined for power-of-two lengths, the implementation pads non-power-of-two arrays up to the next power of two using a sentinel value (`INT_MAX`). For ascending sort, sentinel values migrate to the end; the result is then truncated back to the original length.

A simplified description of the network execution is:

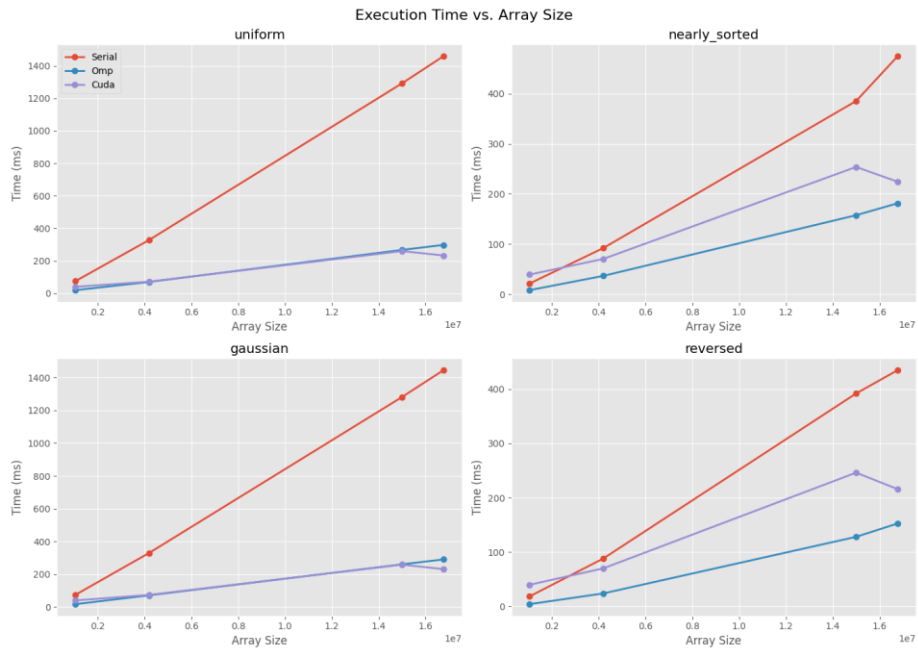
```
paddedSize = next_power_of_two(n)
pad with INT_MAX
copy to device
for k in {2,4,8,...,paddedSize}:
  for j in {k/2, k/4, ..., 1}:
    launch kernel for (k,j) compare-swap stage
copy back
truncate to original n
```

The implementation synchronizes after each stage of launch, which improves determinism and makes the stage boundaries explicit, but can increase overhead relative to more asynchronous GPU pipelines.

6. Results

This section discusses the four figures generated from the measured dataset. The figures are included exactly as produced in the results directory.

6.1 Execution time vs input size



Execution time vs array size.

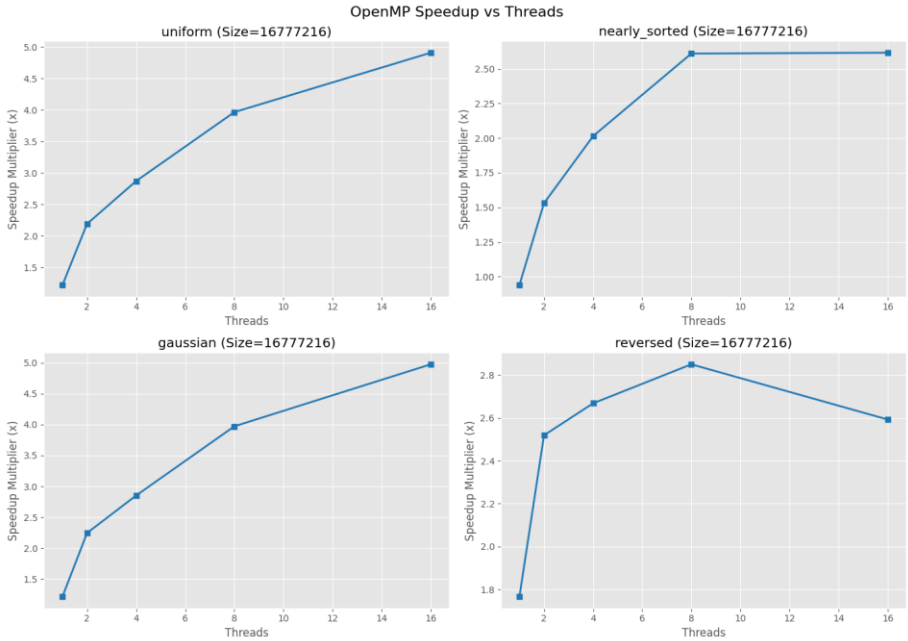
Each subplot compares serial, OpenMP, and CUDA timing for one distribution. The scaling trend is clear: as input size increases, parallel implementations provide substantial gains over the serial baseline.

Headline values for the **uniform** distribution include:

- At **16,777,216 elements**, the serial baseline averages **1459.73 ms**, OpenMP (16 threads) averages **297.37 ms**, and CUDA (block size 256) averages **232.45 ms**.
- At **4,194,304 elements**, OpenMP (16 threads) and CUDA (block size 256) are closely matched at **69.33 ms** and **71.33 ms**, respectively.

These results show that both CPU and GPU parallelism are beneficial at large sizes, and that the GPU implementation achieves the best end-to-end time at the largest tested size on this platform.

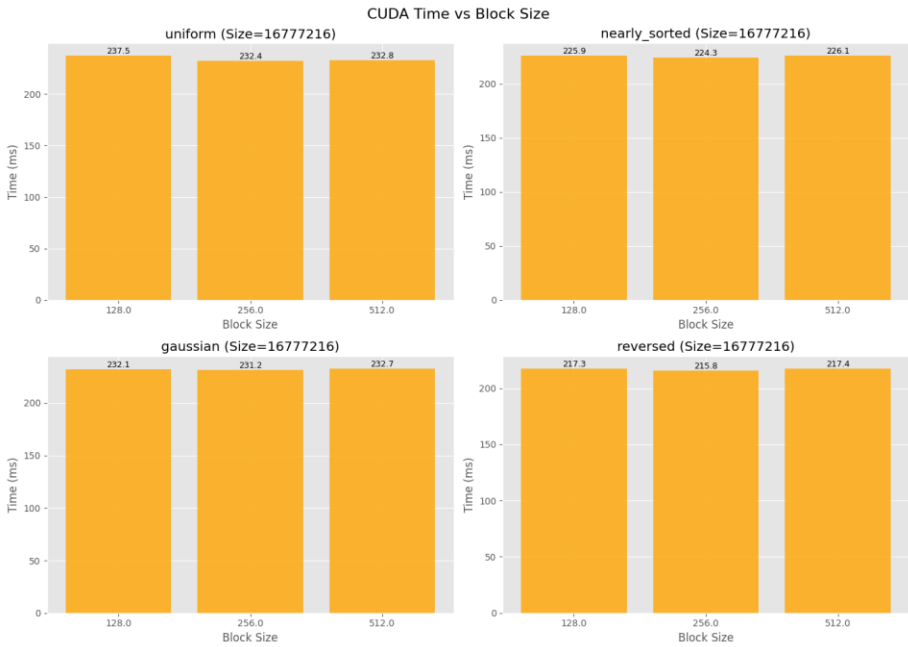
6.2 OpenMP speedup vs thread count



OpenMP speedup vs threads.

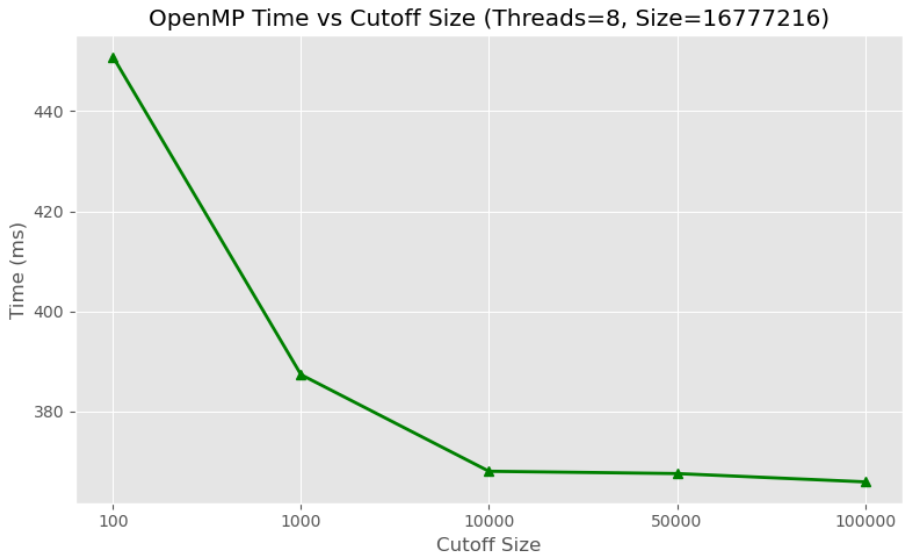
Each subplot uses the largest input size available for that distribution. Speedup improves strongly from 1 to 8 threads and continues to increase to 16 threads.

6.3 CUDA time vs block size



CUDA end-to-end time vs block size.

6.4 Task cutoff tuning for OpenMP



OpenMP runtime vs cutoff threshold.

The cutoff sweep demonstrates that task granularity is critical. At very small cutoffs, excessive task creation increases overhead and hurts performance. As the cutoff increases, runtime decreases and then stabilizes.

7. Discussion

7.1 Scaling and where parallelism helps

The end-to-end results show that parallelism becomes increasingly beneficial as the problem size grows. At the largest size tested, OpenMP reduces runtime by nearly 5× relative to the serial baseline, and CUDA reduces runtime by more than 6×. At mid-range sizes, CPU and GPU solutions can be closed, indicating that the best choice may depend on whether a workload is already resident on the GPU (and can avoid transfers) or naturally CPU-bound.

7.2 Tuning parameters and overheads

Two primary tuning axes emerge:

- **CPU tuning:** thread count and cutoff threshold. Too-small cutoff values can create excessive task scheduling overhead; too-large values can reduce parallelism.
- **GPU tuning:** block size (and, implicitly, kernel launch structure and synchronization strategy). For large sizes in this dataset, block size changes performance modestly, suggesting that end-to-end runtime is influenced by broader overheads as well.

9. Conclusion

This project provides a coherent comparison of sorting across three execution models: optimized serial CPU sorting, OpenMP task-parallel CPU sorting, and CUDA GPU sorting via a bitonic network. The measured results show that OpenMP can deliver substantial speedup when task granularity is tuned appropriately, and that the GPU implementation achieves the fastest end-to-end time at the largest tested size on the measured platform. The included figures make the central tradeoffs visible: CPU scaling depends on task overhead and memory behavior, while GPU performance depends on launch structure, synchronization strategy, and data movement.